

Balancing Coupling in Software Design

SE Radio · Episodio 662 · Vlad Khononov — Arquitecto de software y autor

José Iván Díaz Potenciano · Enol de la Calle Iglesias

¿Qué es el Coupling?

El **coupling** es cualquier dependencia entre dos componentes. Su diseño determina si un cambio en uno obliga a modificar el otro. Existen dos tipos de complejidad: la **esencial** (propia del dominio, no eliminable) y la **accidental** (causada por mal diseño, controlable). El objetivo arquitectónico es gestionar el acoplamiento conscientemente para construir sistemas predecibles, no eliminarlo.

Complejidad y Modularidad

La **modularidad** es el antídoto al acoplamiento excesivo. Dividir el sistema en módulos permite localizar cambios y prever sus efectos. Un módulo bien diseñado cumple tres criterios: **(1)** funcionalidad clara y describible sin conocer su implementación interna; **(2)** interfaz pública estable (API REST, clase pública, firma de método) que actúe como caja negra; **(3)** independencia suficiente para modificarse sin arrastrar cambios en otros componentes. Los módulos existen en todos los niveles —microservicio, paquete, clase, método— formando una estructura fractal.

Historia y Modelos Clásicos

Module Coupling — Yourdon & Constantine (1979)

Define seis niveles de acoplamiento: *Content, Common, Control, Stamp, Data y Functional*. Base teórica fundamental, aunque pensada para sistemas de su época.

Connascence — Meilir Page-Jones (1992)

Refinamiento que mide cuánto dos componentes 'nacieron juntos'. Más expresivo, pero difícil de aplicar en microservicios modernos.

Ambos modelos tienen ~50 años y los sistemas actuales (microservicios, cloud, DDD) los desbordan. Esto motivó a Vlad Khononov —tras el fracaso de un proyecto propio— a proponer **Balanced Coupling**: una herramienta que unifica 50 años de teoría en un framework aplicable hoy.

Tipos de Acoplamiento — Fuerza de Integración

Khononov propone cuatro niveles ordenados de menor a mayor acoplamiento:

- **Por contrato**: solo se comparten interfaces públicas. Máxima independencia (mejor caso).
- **De modelo**: se comparten estructuras de datos entre componentes.
- **Funcional**: se comparte lógica de negocio; los cambios se propagan con más facilidad.
- **Intrusivo**: acceso directo a detalles internos del otro componente (peor caso).

Las Tres Dimensiones del Coupling (Balanced Coupling)

Cualquier dependencia debe evaluarse según tres ejes:

- 1. Integration Strength** — ¿Cuánto conocimiento comparte un componente del otro? A mayor conocimiento compartido, mayor coupling. Ejemplo: el servicio A conoce el esquema interno de la BD del servicio B.
- 2. Distance** — ¿Qué tan lejos están en la arquitectura? A mayor distancia, más caro propagar el cambio. Ejemplo: dos módulos en el mismo servicio tienen distancia baja; dos microservicios distintos, alta.
- 3. Volatility** — ¿Con qué frecuencia cambia el componente? Alta volatilidad + alto coupling = fuente de problemas. Si cambia poco, se tolera más complejidad.

Regla fundamental: **coupling problemático = Strength alta + Distance grande + Volatility alta**. Un caso típico es el **monolito distribuido**: servicios muy separados (distance alta) que comparten demasiado conocimiento (strength alta), el peor escenario posible.

Conclusión

- El coupling siempre existirá; la clave es gestionarlo, no eliminarlo.
- Evalúa cada decisión de diseño desde las tres dimensiones: strength, distance y volatility.
- Los modelos clásicos son válidos, pero Balanced Coupling los actualiza para los sistemas actuales.
- Un buen diseño modular hace que añadir funcionalidades sea sencillo, no un riesgo.